

Triangulation

(A) Introduction:

Involves the measurement of angles in a triangle formed by three points: 2 known points (the 'base' of the triangle), and one unknown point. The location of the unknown point is determined by measuring the angles from the known points to the unknown point, creating a triangle. The position can then be calculated using the angular measurements and the distance between the two known points.

Relies on angle measurements and known distances between reference points.

- Focal length:

<http://www.camera-module.com/product/mipicameramodule/8mp-mipi-camera-module-omnivision-ov8865-sensor.html>

<https://www.mouser.co.uk/new/terasic-technologies/terasic-D8M-GPIO-camera/>

$f = 3.85 \text{ mm}$

- Approach:

Carry out the process in the cloud instead on the hardware

(B) Diagrams / Formulas

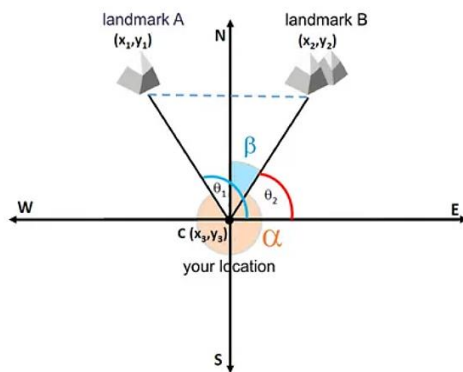


Figure 2: How to triangulate your location from two landmarks.

Let $A(x_1, y_1)$ and $B(x_2, y_2)$ be the two landmark points with bearings as α and β , respectively. If $C(x_3, y_3)$ is our location, the points A , B , and C form the vertices of a triangle, where the angles are:

$$\angle ACE = \theta_1$$

and

$$\angle BCE = \theta_2.$$

We can write the formula for the slope of line AC as:

$$\tan(\theta_1) = \frac{(y_1 - y_3)}{(x_1 - x_3)} \dots (3)$$

Similarly, for line BC, we can write:

$$\tan(\theta_2) = \frac{(y_2 - y_3)}{(x_2 - x_3)} \dots (4)$$

Solving equations (3) and (4) for x_3 and y_3 , we get:

$$x_3 = \frac{(y_1 - y_2) + x_2 \tan(\theta_2) - x_1 \tan(\theta_1)}{\tan(\theta_2) - \tan(\theta_1)}$$

and

$$y_3 = \frac{[y_1 \tan(\theta_2) - y_2 \tan(\theta_1)] - [(x_1 - x_2) \tan(\theta_2) \tan(\theta_1)]}{\tan(\theta_2) - \tan(\theta_1)}$$

NOTES (Report):

<https://www.omnicalculator.com/math/triangulation#triangulation-formula-to-determine-your-position-resection>

Triangulation relies only on angle measurements to locate the unknown point, trilateration uses distance measurements.

Fun fact: While triangulation has an important history in surveying and navigation, GPS uses trilateration to determine your location, not triangulation (the satellites only measure distances)!

(C) Implementation

Let us see how to use the triangulation calculator to determine the location of a landmark. We will consider A as the reference point for our measurement. Let the coordinates of points $A(x_1, y_1)$ and $B(x_2, y_2)$ be $(0, 0)$ and $(5, 0)$, respectively. The measured azimuth or bearings at point A and B be $\alpha = 30^\circ$ and $\beta = 330^\circ$.

1. Using the drop-down menu, select **landmark location**.
2. Enter the **x, y coordinates**, and the **bearings** ($\alpha = 30^\circ$ and $\beta = 330^\circ$) of observation points $A(0, 0)$ and $B(5, 0)$ in the respective fields.
3. The coordinate triangulation calculator will display the **coordinates of the landmark**, i.e., $(2.5, 4.33)$.

```
● root@Shinde:~/Documents/TrEEEvial/vision# python3 triangulation2.py
(2.5000000000000001, 4.330127018922194)
○ root@Shinde:~/Documents/TrEEEvial/vision# █
```

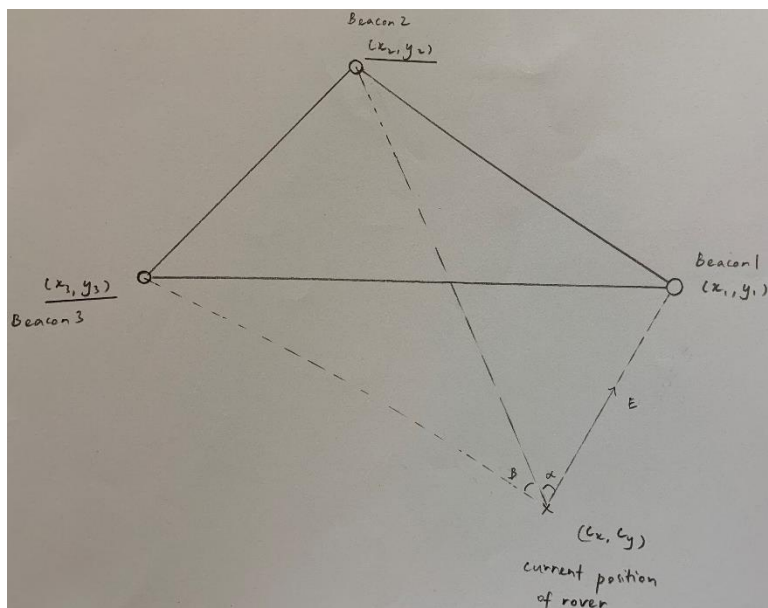
Code works perfectly fine!

Issue:

- How to find the angle from the camera to the beacon? Gyroscope?
Triangulation needs to work with wheel revolution to determine the angles. The rover stops rotating when the camera sees a beacon in the middle section of the frame, hence the amount of angle rotated is the alpha/beta. This process goes on for the 3 beacons, while we only take note of the smaller two angles, and carry out triangulation.
- How to find the coordinates of the beacons?
It will be set by us so that is not an issue.

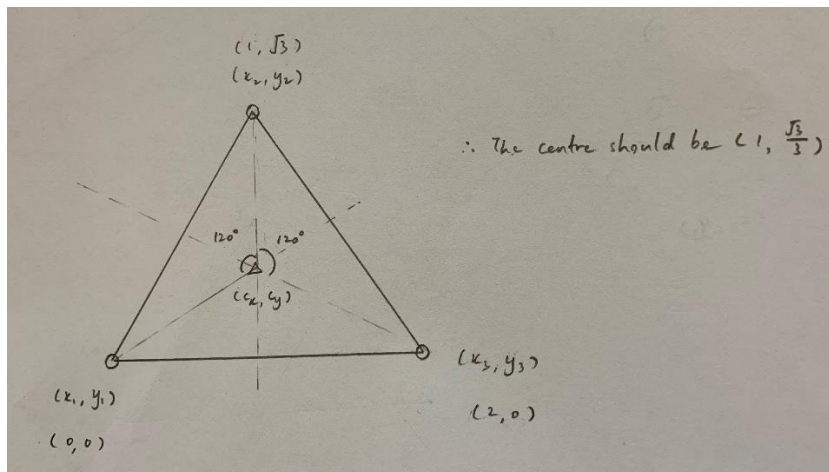
Completed:

Since we have 3 beacons, instead of 2 beacons, we set one of the beacons as the “North” or “East” as described in the diagram below.



Hence, using the formula above, and the coordinates of beacon2 and beacon3, (we do not need beacon1 due to it being treated as “East”), we can calculate the current position of the rover. The angles alpha and beta used, are the two smaller angles among the 3 angles (which sums up to 360) - bearing of beacon1 to the current position, bearing of beacon2 to the current position, bearing of beacon3 to the current position.

Edge case: (angle1 = angle2 = angle3 = 120°)



```

● root@Shinde:~/Documents/TrEEEval/vision# python3 triangulation.py
x3: 0.9999999999999992
y3: -1.732050807568877

```

The edge case does not work, due to the property of tangent function, where $\tan(240^\circ) = \tan(60^\circ)$. Hence, the current position calculated will be below the true current position, i.e. $(1, -1.73)$.

However, this would work for angles - alpha and beta that are less than 180° . Hence, this still requires improvement 😞.